AIGC 5500 | MODULE 06 | 4% OF FINAL GRADE

Lab 06: Fine-Tune with LoRA

Due date posted on Blackboard

Module 06 made four claims about adapting a pretrained model: that a model trained on text can be pointed at a new task by changing only what it is asked to score, that LoRA replaces a full weight update with a low-rank one, that this shrinks the trainable parameters and the optimizer state by orders of magnitude, and that the saving costs some capacity. In this lab you take a real pretrained language model from the Hugging Face Hub, measure it before any training, then fine-tune it fully and with LoRA at several ranks on a small text classification task. You build the LoRA layer yourself, check it against the peft library, and report accuracy, trainable parameters and memory side by side. What is being assessed is whether your implementation is correct, whether your comparisons are controlled, and whether your conclusions stay inside what your measurements show.

Starter code

You are given lab06_starter.py. It downloads the model HuggingFaceTB/SmolLM2-135M (135 million parameters, small enough for a free Google Colab or Kaggle GPU and workable on a laptop CPU), loads a text classification dataset, and provides training, evaluation and memory accounting. Until you write your own, the starter uses peft's LoRA, so every harness runs on day one. You write the answer-only loss, the LoRA layer, the experiments, the memory prediction and the conclusion. Do not edit the starter file directly; copy it into your repository and import from it, or copy what you need into your own work.

There is no new classification head. The model is used as a language model: each example is a prompt that ends in a cue, such as "Text: i feel so happy today\nEmotion:", and the answer is a single label word (sadness, joy, love, anger, fear or surprise). To classify, the starter reads the model's scores for the first token of each label word at the last prompt position and picks the highest. Training uses the ordinary next-token loss on the answer token. Because nothing is added to the model, the same scoring works before any training, and that is your zero-shot baseline. Chance is 1/6 on the emotion task.

```
import lab06_starter as S

model, tok = S.load_model()          # 32-bit weights
train, test, label_ids = S.load_task(
    "emotion", n_train=600, n_test=500, tokenizer=tok)

# Each example: {"prompt_ids": [...], "answer_id": int, "label": int}
acc0 = S.evaluate(model, tok, test, label_ids)  # zero-shot

# One experiment: mode is "none", "full" or "lora".
# Returns a dict: accuracy, trainable, total, weight_bytes,
# grad_bytes, optimizer_bytes, seconds, and more.
info = S.run("lora", seed=0, train_examples=train, test=test,
             label_ids=label_ids, r=8,
             labels_fn=my_answer_only_labels)

# Labels for the loss: -100 means "do not score this position".
labels = S.all_tokens_labels(ids, mask, prompt_lens)    # given
labels = my_answer_only_labels(ids, mask, prompt_lens)  # you write
```

Run lab06_starter.py once, unmodified, to confirm your environment works before changing anything. It prints the zero-shot accuracy, trains a short LoRA run, and reports the trainable parameter count. Expect a few minutes on a GPU and noticeably longer on a CPU. If you are on Colab or Kaggle, choose a GPU runtime before you start and install the requirements with pip install transformers peft datasets.

What to build

Four pieces, in order.

1. **Zero-shot baseline and answer-only loss.** Report the zero-shot accuracy of the pretrained model on the test set and compare it with chance. Then write answer_only_labels(input_ids, attention_mask, prompt_lens), with the same signature as the starter's all_tokens_labels, so that only the answer token is scored. Show with an assertion that exactly one position per example is scored and that no prompt token is. Explain in two sentences what the starter's version trains the model to do with the prompt, and why that is not the skill you are measuring.
2. **LoRA from scratch, verified.** Write LoRALinear(base, r, alpha) that wraps a frozen nn.Linear and adds (alpha / r) B A, with A small random and B zero. Use only tensor operations, nn.Linear and nn.Parameter. Show with code that it equals the wrapped layer at initialization, that its trainable parameter count is r times (in plus out), that a merged plain layer (W plus the scaled product B A) gives the same outputs as the two-path layer after you give B nonzero values, and that it agrees with a peft LoRA layer once you copy the same A and B across. Then print the trainable parameter count and percentage of the whole model for the model with LoRA on q_proj and v_proj at ranks 1, 4 and 16, and show that each equals the formula summed over the layers.
3. **Rank sweep against full fine-tuning.** Using answer-only loss, fine-tune the whole model, and LoRA at ranks 1, 4 and 16 (alpha equal to 2r), with three seeds each (0, 1 and 2) and identical data, steps, batch size and optimizer apart from the learning rate, which you may set per method and must report. Report every seed. Write a prediction in your README before running: will rank 16 match full fine-tuning, and will rank 1 be clearly worse, and why? Then run it and compare. Finally, train rank 16 once more per seed with the starter's all-token loss and report whether scoring the prompt helped or hurt.
4. **Memory: predict, then measure.** For each method in piece 3, predict in bytes the memory held by weights, gradients and Adam state, using 4 bytes per weight and 16 bytes per trainable parameter in 32-bit training (a weight, a gradient and two Adam moments). Write the predictions in a table, then compare them with the measured values from the starter. If you have a GPU, also report the peak memory it recorded and explain why the peak is larger than your prediction. Write a conclusion that says what LoRA saved, what it cost in accuracy, and what your own numbers do and do not establish.

Requirements

Your submission must do all of the following. Each is checked directly by the instructor.

- Write the LoRA layer and the answer-only loss yourself. Calling peft inside your own LoRALinear does not count; peft is there to check against.
- Initialize B to zero and A to small random values, and show that the adapted model starts identical to the pretrained model.
- Compare configurations under identical conditions: the same training examples, test set, steps, batch size and seeds. The only things that may differ between two compared runs are the variable being tested, and the learning rate, which you report.
- Report every seed, not only the mean. Three seeds per cell throughout.
- Do not choose learning rates by looking at the test set. If you tune, tune on a separate split of the training data or on the dataset's validation split, and say which.
- Write your rank prediction in your README before running the sweep, and keep it in the final README even if it turned out to be wrong.
- Quote accuracy relative to the zero-shot baseline and to chance. Use the measurements to argue each claim; do not argue from what the module suggests.
- Report surprising or inconvenient results honestly, such as a seed that fails, a rank that matches full fine-tuning, or full fine-tuning that is worse than LoRA. A null result explained correctly is worth more than a flattering table.
- State the limits of your evidence: one small model, one dataset, a fixed update budget, a handful of seeds, and timings and peak memory that depend on your hardware.

Watch out

Three silent mistakes run without errors and still give plausible numbers. An answer-only mask that is off by one, which scores the last prompt token instead of the answer. A LoRA update that is added to the output but scaled by alpha instead of alpha divided by r, which makes rank 16 train eight times harder than rank 2 for no reason. A comparison where full fine-tuning gets a learning rate that is far too high, because the LoRA rate was reused. Your checks against peft and your small learning-rate search on a validation split are what catch these, so do them before the experiments, not after.

Why a prediction, and why report the failures

A prediction written before the run is the only way to tell whether you understood the mechanism or only explained the result afterwards. The module argues that a task needs a low-rank change and that LoRA at a modest rank can match a full update, while the memory argument is exact. Your measurement decides the first and confirms the second. Whatever you find, report it, and explain it with the mechanism, not with the name of the method.

Repository requirements

The repository is as much a part of the submission as the code. It must contain the following, and it must not contain your virtual environment or any downloaded model weights.

Item	Requirement
.gitignore	Present, committed before your first code commit, and excluding .venv, __pycache__, .env, saved model files and any large output files
README.md	Explains the task, each of the four pieces, your rank prediction, your results tables, and how to reproduce your results, including which hardware you used
requirements.txt	Generated with pip freeze, including torch, transformers, peft and datasets
Your code	Your answer-only loss and LoRA layer, the verification against peft, the experiments and the memory accounting, as scripts or a single notebook, together with lab06_starter.py
Figures	At least one figure saved as an image and referenced from the README, for example accuracy against trainable parameters for each method
Commit history	At least three commits, each a coherent change, with messages that describe what changed

README template

You may use this structure directly. Replace the content, not the shape.

```
# Fine-Tune with LoRA

A pretrained 135M language model used as a classifier, adapted fully and
with LoRA at several ranks, compared on accuracy, parameters and memory.

## Setup
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt

## Run
python loss_and_baseline.py   # piece 1: zero-shot, answer-only loss
```

```
python lora_layer.py          # piece 2: my LoRA layer, checks against peft
python sweep.py               # piece 3: full vs LoRA r = 1, 4, 16
python memory.py              # piece 4: predicted vs measured bytes

## Hardware
GPU or CPU used, and the total time for the full set of runs.

## Prediction (written before running the sweep)
What I expected for rank 16 and rank 1 against full fine-tuning, and why.

## Results
Zero-shot accuracy against chance.
Verification: what matched peft, and the parameter counts.
Sweep: accuracy per seed for each method, trainable parameters, percent.
All-token against answer-only loss at rank 16.
Memory: predicted and measured bytes per method.

## Conclusion
What LoRA saved, what it cost, and what these measurements
do and do not establish.

## Known limitations
Anything you are aware of that does not work, or that you would improve
with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 06 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

Instructor GitHub account

`hossein.pourmodheji@humber.ca`

Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that the instructor cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Before you submit, run this check

Clone your own repository into a different folder, create a fresh environment, install from requirements.txt, and reproduce every table and figure following only your README. The full set of runs is sixteen trainings of 135 million parameters. On a free GPU this takes well under an hour; on a laptop CPU the starter's default budget takes roughly one to two hours, so start early. If you must, reduce the number of training steps or the training set, and say so in your README. If everything completes cleanly and agrees with what you reported, your submission is complete.